

THE3Report

1st İlkim Oğul
CENG
METU
223767

2nd Onur Yaman
MATH
METU
2007961

I. FACE DETECTION

A. The K-means algorithm

We initially tried to follow the K-means algorithm provided by Lecture Notes 8, but it either took unfathomably long, or I merely failed to exit the $C(i)=C(i+1)$ loop, so we had to give up (which truly drove me insane after having to suffer so much nonsense from Numpy) and use internet sources for some method.

B. Labels

Following the Face Detection steps given in LN8, I picked approximate RGB values for faces, shadowed faces, beards (there were very few beards so I used that one for secondary non-face areas), and non-faces for each input image by using a pixel-picker. Which would then be compared to the cluster results from the K-means algorithm.

But we had to drop all that completely as after 10 minutes my abominable creation is still stuck in some loop.

Finally we decided to add this work in the file we will upload. However, we used Haar Frontal Face Cascator in order to obtain the results you desire. So we provided "haarcascade.frontalface.default.xml" file which belongs to "cv2". Your file path should contain this file in order to get desired outputs corresponding to face detection and circling faces.

C. Color Regions that could have been

Never actually got to this step. Had some ideas (mostly different variations of manual counting, which had plenty trauma attached), but poor time management resulted in a method that merely censored faces and some other sections with bright red. Or would have done so, if the thing actually worked.

D. Note

We will include the failed code as the3failed, as proof of work or something. It just took too much out of my life to be simply deleted.

II. PSEUDO COLORING

A. Getting Started, and Guesswork

Pseudo-coloring is a technique where we map each of the grey levels of a grayscale image into an assigned color. When we process this technique, we get a pseudo-colored image that generally contains more information than previous one.

Our initial thought process here is that we'll be approximating manually-inputted hues of skintones by processing the greyscale values. Thankfully, instead of approximating rgb values manually, we were given color bars to pick from.

B. Following the Algorithm

- Read images as RGB
- Convert RGB to LAB
- Calculate mean and standard deviation of LAB images
- Ensure that standard deviations are non-zero
- Subtract mean of target from target
- Scale target using standard deviations
- Add mean of source to target
- Convert LAB to RGB

where target is the grayscale image, source is given image to color the grayscale image.

C. The Process

We adopted online resources to be able to implement `colormages()`. In addition, it requires some helper functions such as `rgb2lab` and `lab2rgb` so that we can apply color mapping in LAB color in a more convenient way. Thus, we go through each pixel in the grayscale image and we select the best matching sample in the source image by using neighborhood statistic with the help of `imgstats` function.

Then we transferred color content from source color images to grayscale images. As seen in Outputs folder, in four images, the technique works well in terms of the idea of pseudo coloring. By `imgstats` function, we can measure mean and standard deviation of source image. So we suppose that we are using average color content of source image. However, this functionality could be improved and better results could be achieved (pixel by pixel determination by using more advanced statistical methods), but this is the subject of machine learning and is beyond the scope of this course. Our aim here is to give a model answer.

We used `skimage's filters.sobel` for edge detection. As stated in our lecture notes 5, Sobel filter is a kind of gradient filter. It combines the edge maps, obtained at the output of gradient filters in vertical and horizontal directions to define the magnitude and directions of edge pixels.

References: <https://github.com/prasunroy/color-transfer> and

the paper "Color Transfer Between Images" by Erik Reinhard, Michael Adhikhmin, Bruce Gooch and Peter Shirley (2001)



Fig. 1. Image 1, This is the target image



Fig. 4. Final version after transforming the image to RGB space obtained by transferring colors of 3source onto img 3

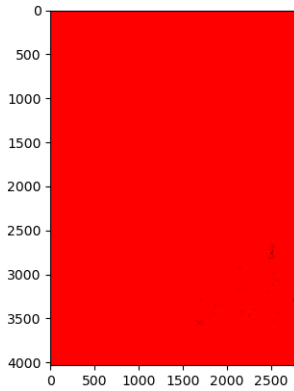


Fig. 2. This is LAB Transform of target image.



Fig. 5. This is the gradient of the final version

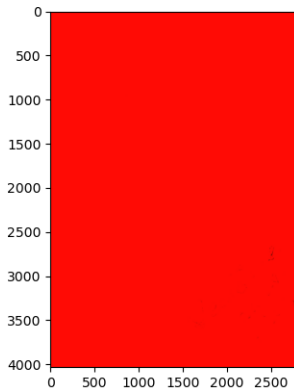


Fig. 3. After Standard Deviation, Mean calculated and corresponding mathematical operations applied on LAB color space

D. Observations

It is clear that HSI images are brighter than RGB images. One on hand, HSI is based on Human Visual System. On the

other hand, RGB is useful to perform mathematical operations in terms of time complexity. So we are reading images as RGB.

We provided a function called, `rgb2hsi`, to get the desired images by using online resources. $O(n^2)$, so it is quite slow.